

Buy or Wait? — Interview Preparation

HackerRank Orchestrate, September 2026. An AI financial decision agent that reads nine CSVs of user financial history plus untrusted messages and images, and for each of 250 purchase requests decides whether to pay in full, pay partially, use an installment option, wait, or not proceed.

The 30-second answer to “what did you build?”

“It reconstructs each user’s cash position as of their request date, projects 90 days forward from their recurring commitments and confirmed income, then asks two questions exactly: how much is safe to pay today, and what is the earliest date a full payment is safe. Safety means the projected balance never dips below the user’s stated minimum — not on the payment day, but at every trough after it.

The surprise is that it is barely an LLM problem. Template matching classifies 100% of the 215 messages and explanations are deterministic string fills, so per-request model cost is zero. The only model spend is four image reads, done once and cached: \$0.046 total.”

1. The numbers — cheat sheet

Every figure below was independently re-derived by `audit/verify_claims.py` (32 CONFIRMED, 5 CODE-DEPENDENT, 0 WRONG). Quote these with confidence.

Submission

Artefact	Value
output.csv	250 rows, 0 validator violations, SHA-256 9547861f...50ea
code.zip	54 files, ~285 KiB, secret scan clean
chat_transcript.md	682,708 bytes; 29 prompts, 215 replies, 479 tool calls
Tests	345 passing
Runtime	~1 second for all 250 rows, no network, no API key needed

Accuracy against the 25 labelled sample rows

Column	Exact	Note
amount_safe_to_pay	2 / 25	16 over-state, 7 under-state — the known defect
affordability_status	18 / 25	
recommended_payment_method	20 / 25	20/20 when the labelled method was generated at all
payment_plan	18 / 25	
earliest_date_for_full_payment	13 / 25	
spending_changes_needed	22 / 25	
decision_explanation	12 / 25	
All columns	2 / 25	MARE on amount_safe_to_pay: 1.248

Status distribution over the 250 evaluation rows

Status	Count	Share	Reference	Delta
affordable_now	65	26.0%	12%	+14.0 pts ← the residual defect
affordable_with_plan	69	27.6%	36%	−8.4
affordable_later	45	18.0%	24%	−6.0
not_affordable	71	28.4%	28%	+0.4

Cost and model usage

Metric	Value
Model calls per request	0

Metric	Value
Tokens per request	0
Total model calls for the whole run	4 (one-time image prepass)
Total tokens	7,630 (7,240 in / 390 out)
Total cost	\$0.046
Model pinned	claude-opus-5, temperature 0.0, top_p and seed unset

2. Architecture — the data flow in five lines

Stage	What happens
dataset/ → load.py	Nine CSVs become frozen dataclasses. Money is Decimal parsed from source text, never float. Foreign amounts convert at the settlement-date rate.
→ state.py	~25,000 events land in exactly one of nine cash buckets. Repeating costs group into series with an inferred cadence. Lifecycle collapsing via linked_event_id. current_available_balance is truth as of request_date.
→ extract.py + forecast.py	Messages become typed facts; facts amend the projection; a daily balance walks forward 90 days. Conflicts resolve by explicit precedence.
→ capacity → candidates → changes → rank	How much is safe, which plans are eligible and safe, which spending changes rescue an unsafe one, and which plan wins on five strict criteria.
→ render.py → validate.py	Eight formatted columns, re-checked from the written CSV text before the file is accepted.

src/config.py holds every decision that could reasonably have gone another way, each with the measurement that set it.
src/run.py wires it together.

3. The decisions that define the system

a) Safety is a property of troughs, not payment days

A payment of P on day d lowers the projected balance on every later day by exactly P — it shifts the tail down, it does not reshape it. So the tail stays above the floor precisely when $\min(\text{balance over } [d, \text{end}]) - P \geq \text{minimum}$. The binding constraint is almost always the pre-salary dip, not the payment day. A test builds a payment leaving three times the minimum on the day itself, which still breaks the floor 70 days later.

b) The task looks like an LLM problem and mostly is not

40 regex/template patterns classify 215 of 215 messages. The LLM fallback is implemented, tested, and never fires. Explanations are six deterministic templates, not generated prose — a test asserts render.py imports neither the Anthropic SDK nor the metered path. That is the headline efficiency result, not dead code.

c) Description-keyed series — right for expenses, wrong for income

The rulebook keys recurring series by description, which is what makes the labelled spending-change rows reproduce. But one income stream is often recorded under several payer names, and description-keying shattered it into four irregular series so no income projected at all. Fixed by aggregating salary credits into one stream as a fallback, only when no description-keyed salary series was found — so users with consistent payroll keep validated behaviour.

d) Untrusted content fails closed rather than being detected

Exactly one template flags is_instruction_attempt, and it fires on 2 of 215 messages. Any other directive matches no template and becomes UNKNOWN with affects_cash = False. Nothing reaches a forecast except a typed fact with affects_cash = True, so unrecognised text is inert by construction. The flag is an audit label, not the defence.

e) Two metrics disagreed and the noisy one lost

Sweeping the staleness gate, exact-match count peaked at one setting and continuous error at the opposite one. MARE moved monotonically by 43% while exact-count spread over 1–3 rows out of 25. The continuous metric decided, and request_10's 13.5× overstatement disappeared. This rule — continuous over discrete — then governed every later sweep.

4. How AI was used — answer this honestly

The honest shape of it

Claude Code (Opus 5) wrote essentially all of the code, across a ~17-hour session of 29 prompts. The transcript shows it. That is what the tool is for and the challenge explicitly permits any AI assistant.

What I contributed was direction and adversarial pressure. Each prompt set a phase, its constraints, and — critically — a stopping condition: measure before you build, report before you change a default, stop and report after each step. Several prompts were explicitly designed to catch the model overclaiming.

Inside the product, AI does almost nothing. 4 image reads, cached. Zero model calls per request. I deliberately did not add an LLM prose pass to explanations, because determinism was worth more than fluency.

The specific prompting patterns that mattered

- **Measure-before-build gates.** extract.py had to hit ~85% template coverage before the LLM fallback could be written. It hit 100%, so the fallback was never needed.
- **Report-before-change.** Calibration sweeps had to report findings and stop, not batch changes. Three global levers were measured and rejected on that rule.
- **Contradiction over compliance.** The brief said ‘where an instruction conflicts with the data, report it rather than resolving it silently.’ That surfaced roughly eight places where my own premise was wrong — e.g. I assumed 0 blank amounts needed images; it was 4.
- **Adversarial self-review.** A late prompt asked the model to read the codebase cold as a hostile interviewer and find where the author overclaims. It found three real documentation defects, all since fixed.

Where AI got it wrong, and how it was caught

Failure	How it surfaced	Fix
Fabricated ‘time remaining’ across nine turns; invented log timestamps	I challenged the figure directly	Appended a correction entry to log.txt rather than rewriting history; transcript now generated mechanically so timestamps are the harness’s own
Claimed both capacity answers were ‘closed-form’	Adversarial review pass	Only amount_safe_to_pay is. The date is an exact forward scan resting on suffix-minimum monotonicity. Corrected in header and docs
Docs described directive detection that did not exist	Adversarial review pass	Rewritten to describe the fail-closed mechanism that actually runs
Overclaimed a sweep result (‘MARE under 1.5 □ affordable_now 62–66’)	Final verification pass found four counterexamples	Replaced with a total-deviation framing that admits no exceptions; the original error recorded in the log as a CORRECTION block

This is a strength, not an embarrassment. The system that caught each of these — an independent claim verifier that re-derives every numeric claim from the raw CSVs rather than by re-running the producer — exists precisely because the first fabrication reached a submission artefact.

5. Known limitations — lead with these, do not wait to be asked

The single weakest point

The forecast over-states spending capacity. affordable_now is 65 against a reference of 30 (+14 points) and amount_safe_to_pay is exact on only 2 of 25 labelled rows, over-stating on 16.

Why it is a documented limitation rather than a defect: it is measured, attributed per request in audit/capacity_trace.jsonl, bounded in the README, and the obvious fixes were tried and rejected with numbers — orphan-expense aggregation overshoot to a 1.41 run-rate ratio and collapsed status accuracy from 18/25 to 9/25. Two rows, request_05 and request_21, remain unexplained and are named as such.

What I would do with more time: the residual is not reachable through recurrence detection — 22 settings across four constants were swept and none moved it. That narrows it to capacity or candidate generation, which is where I would look next.

Other limits, stated plainly

Limitation	Honest framing
100% template coverage is fitted, not held out	The templates were written with all 215 messages visible. Coverage on unseen phrasing is unknown and could be materially lower. That is exactly why the LLM fallback exists and stays tested, and why unmatched text fails closed.
Ranking criteria 3–5 never fire on real data	Zero labelled rows have more than one eligible candidate. 271 of 275 requests are settled by having 0 or 1 candidate; only 4 reach a tie-break, all criterion 2. '20/20' means ranking never mis-picked when offered a choice — it does not mean the tie-break ladder is validated.
Injection property has narrow effective coverage	The scan covers all 215 messages so a flagged one cannot be missed, but only 2 ever flag — so effective coverage is 2 messages across 2 users. What covers the other 213 is the structural fail-closed property, not this test.
Spending changes under-generate: 3 of 250	Against roughly 30 expected. That is a capacity problem upstream, not a selection problem. Given the labelled deficit the rule reproduces 2 of the 3 labelled rows; end to end it reproduces 0 of 3, because the forecast finds no deficit for any of them.
4 of 16 images are read	The other 12 blank-amount events settled before their request date, so they are already inside current_available_balance. Verified that all 12 are settled pre-request, so the omission is provably output-neutral — but it is an inference, not something the spec licenses.
request_11 contradicts the rulebook	Its labelled spending change targets a non-recurring two-occurrence series, which the rulebook's own recurring-only rule forbids. Reported, not resolved.

6. Hard questions — and the honest answer to each

Q. You say capacity is closed-form. Walk me through earliest_date_for_full_payment.

One is closed-form: `amount_safe_to_pay` is a single subtraction off the suffix minimum, no iteration. The other is not — it is an exact forward scan over at most 91 days. It is correct because the suffix minimum is monotone non-decreasing in the start date, so once the predicate turns true it stays true and the first success is provably the earliest date. Exact, but searched. My module header used to say both were closed-form; that was an overstatement and I fixed it.

Q. How would you detect an injection that isn't a prize-fee scam?

I wouldn't detect it — I'd fail closed. Unrecognised text produces an UNKNOWN fact with `affects_cash` False, which no forecast will read. That is the right safety property, and it is stronger than detection because it does not depend on my having anticipated the phrasing. My docs used to describe detection I don't have; that is corrected.

Q. 20/20 on ranking. On how many rows did ranking actually choose?

None of the labelled ones. Zero labelled rows have more than one candidate. 20/20 means ranking never mis-picked a lone candidate. Across all 275 requests, only 4 reach a tie-break and all 4 are decided by criterion 2, so criteria 3, 4 and 5 have never executed on real data. The header says so, but the headline number reads stronger than it is.

Q. Why is CADENCE_CONSISTENCY_THRESHOLD 0.6?

Originally because a synthetic irregular-gap test needed it. That was the honest answer until the last pass, when I swept it — 0.0 through 1.0, holding the other three cadence constants fixed. It sits on a cliff, not a plateau: moving 0.5 → 0.6 takes status exactness from 9/25 to 18/25 as the recurring-series count halves. 0.7 wins MARE by 3% but collapses the outflow run-rate ratio from 1.00 to 0.84, a 16% systematic loss in the unsafe direction. Confirmed at 0.6, and the sweep is recorded at the constant's definition.

Q. Did you ever change a default just because the exact-match count improved?

No, and I had one genuine temptation. `MAX_SKIPPED_OCCURRENCES` = 2 beats 3 on MARE by 2.8% and on every exact column. But the entire gain is one row — `request_09` going from 0.923 relative error to exactly 0.000 — while the run-rate ratio falls 1.00 → 0.94 across all 25 users and the over/under split moves 16/7 → 17/5. It buys one fixed under-statement by pushing the whole dataset further into over-statement, which is both the unsafe direction and the known residual defect. Rejected, and the rejection is logged.

Q. What is the biggest thing you would change if you started over?

I would build the claim verifier first, not after a fabricated figure reached a deliverable. Everything downstream — the as-of markers on superseded numbers, the PINNED MEASUREMENT docstring convention, generating the transcript mechanically — came from that one failure. Having it from turn one would have cost an hour and saved several.

Q. Where is the LLM actually doing work?

Four image reads, resolving blank-amount events that had not yet settled. That is it. The prepass is cached to `audit/image_cache.json` with per-entry provenance, so the scored run makes zero calls and is reproducible without an API key. One of those four — `event_1786` — is a good story: my brief described it as a pending credit, the ledger says debit, and resolving it correctly moved that row's error down by exactly 118.00.

Q. Why Decimal everywhere instead of float?

Money. A float cannot represent 0.1 exactly, and the whole system turns on whether a projected balance sits above or below a floor — a half-cent bias in a consistent direction is a safety bug, not a rounding nit. Amounts are parsed from source text, so `Decimal('603.3')` not `Decimal(603.3)`, and a test asserts no float appears in any money path. Safe amounts round DOWN, never half-up, because rounding up could push the balance below the floor the figure was computed to respect.

7. Quick reference — config and where things live

Constant	Value	Why
SERIES_STALENESS_PERIODS	12.0	P11-S2 sweep; was 4.0, which dropped live series
MIN_SERIES_OCCURRENCES	2	Under-detecting an expense is unsafe; income is safe
CADENCE_CONSISTENCY_THRESHOLD	0.6	P11-S6 sweep; sits on a cliff
CADENCE_SKIP_WEIGHT	0.5	P11-S6; run-rate ratio crosses 1.00 exactly here
CADENCE_TOLERANCE_DAYS	1	P11-S6; real debits drift a day at month boundaries
MAX_SKIPPED_OCCURRENCES	3	P11-S6; 2 wins on labels but loses 6% of outflow
FORECAST_HORIZON_DAYS	90	Spec
DEFAULT_SALARY_DAY_OF_MONTH	15	Fallback only when history gives no anchor
FX_DATE_KEY / QUANTIZE_AT	cash_date / conversion	Measured INERT on this data; kept because another dataset would separate them

Where to point the judge

File	What it holds
docs/reading_order.md	What to read first and why; the three interesting decisions
src/config.py	Every calibrated constant with the sweep that set it and the rejected alternatives
evaluation/calibration_log.md	Full Phase 11 history, dated, with as-of markers. Current figures live only in the FINAL SCORECARD at the foot
audit/verify_claims.py	Independent re-derivation of every numeric claim; 32 CONFIRMED, 0 WRONG
docs/worked_example.md	The one model-touched step, end to end
evaluation/usage_report.md	Token and cost accounting

Three things to say out loud early in the interview

1. **“The per-request model cost is zero, and that was the design goal, not an accident.”** It reframes the whole submission.
2. **“The forecast over-states capacity by 14 points on affordable_now, here is the measurement and here are the three fixes I tried and rejected.”** Leading with the known defect buys credibility for everything else.
3. **“I had the model adversarially review its own codebase and it found three places I had overclaimed — all fixed, all recorded rather than edited away.”**

Generated from verified figures only. If asked something not in here, say so and offer to re-derive it — every number in the project is reproducible with `py audit/verify_claims.py`, `py evaluation/calibrate.py`, or `py -m evaluation.sweep_cadence`.